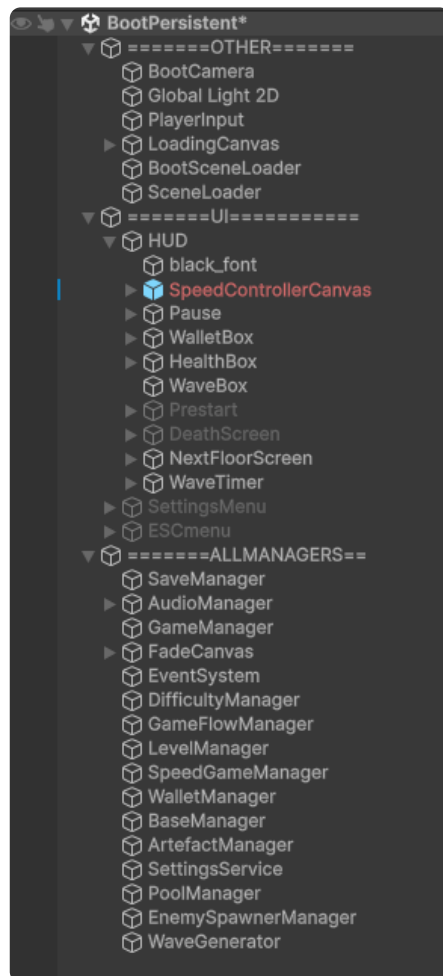




BootPersistent Scene: Global Managers, HUD.

BootPersistent.



BootPersistent.

При разработке игр ранее я делал системы немного проще и не думал о глобальном масштабе. Начав делать TDI я узнал, что менеджеры можно не просто делать DontDestroyOnLoad, но и держать их на специальной сцене, которая никогда не будет закрываться. Более того - инвентарь, специальные экраны (экран смерти, экран победы) можно также держать на этой сцене и открывать в любой момент поверх игрового уровня, тем самым, не пересоздавая их, не инициализируя каждый раз данными из менеджеров, и не заставляя GC стирать объекты.

Boot сцена обычно служит только загрузчиком. Можно было сделать её отдельной и всё равно инициализировать менеджеры в ней, благодаря

DontDestroyOnLoad они также жили бы между сценами.

В таком случае в Persisten я бы хранил глобальные штуки, но не ставшие менеджерами, тот же HUD и ключевые экраны (DeathScreen, NextFloor, Prestart).

- ✓ Я просто объединил сцену загрузки и персистент (постоянную) в одну сцену. Тут у меня лежат все менеджеры, которые в любом случае становятся DontDestroyOnLoad, лежит HUD, и прочие глобально доступные объекты.

BootSceneLoader.

- ⚠ Информация ниже уже была указана на странице Scenes System.

- i Одноразовый скрипт отображения загрузочного экрана и загрузка MainMenu сцены.

```
1 using System;
2 using System.Collections;
3 using UnityEngine;
4 using UnityEngine.SceneManagement;
5 using UnityEngine.UI;
6 public class BootSceneLoader : MonoBehaviour
7 {
8     [Header("UI References")]
9     [SerializeField] public Slider progressBar;
10    [SerializeField] public Image background;
11    [SerializeField] private string startScene = "MainMenu";
12    [Header("Settings")]
13    [SerializeField] private float minLoadTime = 1.5f;
14    [SerializeField] private float smoothSpeed = 0.03f;
15    [Header("ClearObjects")] [SerializeField]
16    public GameObject[] clearObjects;
17    void Start()
18    {
19        StartCoroutine(LoadMainScene());
20    }
21    // Начинаем асинхронную загрузку стартовой сцены, в нашем случае - Main.
22    // Пока загрузка идёт вычисляем положение слайдера.
23    private IEnumerator LoadMainScene()
24    {
25        AsyncOperation op = SceneManager.LoadSceneAsync(startScene, Loa
26        op.allowSceneActivation = false;
27        float timer = 0f;
28        float targetProgress = 0f;
```

```

29     float displayedProgress = 0f;
30     /*
31     sceneProgress содержит текущий прогресс загрузки сцены.
32     Операция загрузки возвращает 0.0 - 0.9 (90%) - как результат считывания
33     с диска и загрузки в оперативную память. Вот только мы так и не получим
34     в виде 1, пока не активируем сцену вручную.
35     А автоматическую активацию мы выключаем - op.allowSceneActivation = false
36     Это сделано на случай, если сцена загрузится слишком быстро,
37     а у нас есть ещё один прогресс. Когда мы поделим 0.9\0.9, то получим 1.
38     */
39     while (!op.isDone)
40     {
41         timer += Time.deltaTime;
42         float sceneProgress = Mathf.Clamp01(op.progress / 0.9f);
43         float timerProgress = Mathf.Clamp01(timer / minLoadTime);
44         // Выбираем то, что дальше (грузим минимум minLoadTime)
45         targetProgress = Mathf.Max(sceneProgress, timerProgress);
46         // Плавно приближаем отображаемое значение
47         displayedProgress = Mathf.Lerp(displayedProgress, targetPro
48         progressBar.value = displayedProgress;
49         // timerProgress через деление time \ minLoadTime переводит секунды в п
50         // загрузки:
51         // 1\1.5 = 0.6        1.5\1.5 = 1        2\1.5 = 1.3
52         // Проверка на завершение
53         if (sceneProgress >= 1f && timerProgress >= 1f)
54         {
55             yield return new WaitForSeconds(1f);
56             op.allowSceneActivation = true;
57             while (!op.isDone)
58                 yield return null;
59         }
60         // Слайдер работает с процентами в виде 0-1. Поэтому 1.3 = выходу за гр
61         // С помощью Mathf.Clamp01 мы гарантировано будем возвращать не более 1
62         // Я сделал искусственную загрузку длиной в 1.5 секунды + 1 секунда,
63         // когда слайдер дойдёт до 100%.
64         yield return null;
65     }
66     SceneManager.SetActiveScene(SceneManager.GetSceneByName(startSc
67     foreach (var clearObject in clearObjects) { Destroy(clearObject
68     Scene mainMenuScene = SceneManager.GetSceneByName(startScene);
69     SceneLoader.Instance.SetCurrentScene(mainMenuScene);
70     AudioManager.Instance.PlayMusicType(MusicType.Menu);
71 }
72 }

```

Также укажу, что мы используем здесь `Mathf.Lerp` - известный и простой способ создания плавности. Формула: $A + (B - A) \times t$.

Берём точку старта A, точку куда нужно дойти B, и t - процент расстояния от A и B, который нужно пройти за один текущий кадр.

$t = 0.3 \times 0.016 \times 60 \sim 0.288$ (28.8% от оставшегося)

Допустим начало: $0 + (0.66 - 0) \times 0.3 = 0.198$ (20%).

Следующий кадр: $0.19 + (0.66 - 0.19) \times 0.3 = 0.33$ (в этом примере B ещё не успел поменяться)

Это лишь пример, потому что цикл `while` идёт куда чаще (в примере 0.66 означает target за секунду). На деле мы будем прибавлять по крошечным процентам каждую долю

секунды, из-за чего получим плавную загрузку, которая местами всё же может быть быстрой.

Если пойти по примерам дальше, то мы увидим, что каждый следующий прыжок значения всё меньше, чем предыдущие. Это называется асимптотическим приближением.

P.s. расписал это больше для себя и для своего углублённого закрепления, чем для самой документации, но почему нет 😊

Global Managers.

i Менеджеры работают по паттерну Singleton. Очень часто его называют анти-паттерном, но в сфере разработки игр он чаще необходим. Да, с его тестированием могут возникать проблемы и его тяжелее отлаживать, но чем меньше проект - тем проще.

= Идея Singleton (для маленьких) - есть некий класс, который существует в одном экземпляре и через статическое поле доступен в любом месте программы. В C# мы устанавливаем приватный конструктор, чтобы класс сам мог создать себя один раз, а доступ к нему открылся через статическое поле. В Unity мы хотим держать менеджер как объект на сцене, поэтому конструктор нужно оставить. Используется другой способ для контроля единого экземпляра:

```
1 using UnityEngine;
2
3 public class Manager<T> : MonoBehaviour where T : MonoBehaviour
4 {
5     private static T _instance;
6     public static T Instance => _instance;
7
8     protected virtual void Awake()
9     {
10         if (_instance != null && _instance != this)
11         {
12             Destroy(gameObject);
13             return;
14         }
15
16         _instance = this as T;
17         DontDestroyOnLoad(gameObject);
18     }
19
20     protected virtual void OnDestroy()
21     {
22         if (_instance == this)
23         {
24             _instance = null;
25         }
26     }
27 }
```

Мы создаём два статических поля - одно приватное, другое публичное, для доступа. В Awake мы проверяем, если текущий менеджер не пустой, и не равен этому объекту, то мы удаляем этот объект. Это небольшая защита, чтобы избежать дублирования менеджеров. Иногда она может сработать, вызвав некорректное поведение. Чаще вам просто самим нужно понимать, что менеджер лежит либо в единичном экземпляре (как у меня), либо пересозадаётся, корректно соблюдая пересоздание. Я создаю менеджеры лишь единожды, а значит такая проверка спасёт меня в случае, если объекта на сцене будет два. Один из них будет уничтожен и после никаких проблем не будет.

Если же проверка не пройдена, то значит менеджера нет. Присваиваем ссылку на данный экземпляр в статическое, доступное глобально, поле `_instance`. Мы не сможем сами в него присвоить менеджер извне, потому что оно приватное, а работаем мы с ним через свойство `Instance`.

Делаем объект неуничтожаемым при смене сцены (хотя в моём случае это не обязательно, `Persistent` сцена живёт всегда).

Теперь любой менеджер просто наследует от этого класса:

```
1 public class HudManager : Manager<HudManager>
2 {
3     ...
4
5     protected override void Awake()
6     {
7         base.Awake();
8
9         defeatScreen = defeatScreenCanvas.GetComponent<DefeatScreen>();
10        prestartUI = prestartCanvas.GetComponent<PrestartUI>();
11        nextFloorScreen = nextFloorCanvas.GetComponent<NextFloorScreen>();
12        if (waveHud == null) { waveHud = GetComponent<WaveHud>(); }
13
14        Hide();
15        HideDefeatScreen();
16    }
```

Если метод `Awake()` нам нужен, то мы переопределяем его, прежде всего указав `base.Awake()`, чтобы статический доступ работал как задумано. Теперь мы можем просто `HudManager.Instance.Method()`;



Повторюсь, в том же бекенде синглтон - антипаттерн. Его сложно тестировать, у него могут быть проблемы с многопоточностью и гонкой данных. В геймдеве дела с ним обстоят немного проще. Однако и в геймдеве от них отказываются, когда речь идёт о больших проектах или онлайн-режимах.

Заменить синглтон можно DI-контейнером. Вместо того, чтобы менеджер сам объявлял себя главным и доступным, некий специальный сборщик сам подсовывает нужные сервисы туда, где они требуются.

У меня простой PET-Project (UltraDemo) игра, и мне в целом пока достаточно довести игру до конца с простой системой менеджеров и глобальными доступами. В следующих проектах я, конечно, попробую системы поинтереснее и серьёзнее.

DI контейнеры.

Данный блок я запишу в том числе для себя.

Вместо глобального менеджера со статическим доступом в едином экземпляре, мы делаем обычный класс:

```
1 using UnityEngine;
2
3 public class WalletManager : MonoBehaviour
4 {
5     public int Coins { get; private set; }
6
7     public void AddCoins(int amount)
8     {
9         Coins += amount;
10        Debug.Log($"Монеты обновлены: {Coins}");
11    }
12 }
```

Допустим, в своём HudManager ранее я просто делал так: `WalletManager.Instance.Coins`; Теперь сделаем иначе:

Zenject - популярный DI фреймворд для Unity. Мы просто указываем атрибут `[Inject]` над нужным методом и он сам принесёт сюда ссылку.

```
1 using UnityEngine;
2 using Zenject;
3
4 public class HudManager : MonoBehaviour
5 {
6     private WalletManager _wallet;
7
8     [Inject]
9     public void Construct(WalletManager wallet)
10    {
11        _wallet = wallet;
12    }
13
14    public void UpdateUI()
15    {
16        Debug.Log($"Отображаем в HUD монеты: {_wallet.Coins}");
17    }
18 }
```

Теперь нужен тот, кто принесёт ссылку (объект) - нам нужно создать специальный объект инсталлер и положить его на сцену:

```
1 using Zenject;
2
3 public class GameInstaller : MonoInstaller
```

```

4 {
5     public WalletManager walletManagerPrefab;
6
7     public override void InstallBindings()
8     {
9         Container.Bind<WalletManager>().FromComponentInHierarchy().AsSi
10    }
11 }

```

Метод `InstallBindings` переопределяется и мы дописываем ему инструкцию: если кому-то с атрибутом `Inject` понадобится экземпляр определенного класса, то мы создадим его и выдадим ссылку на этот объект.

На деле это очень похоже на синглтоны моей `Persistent` сцены. Потому что `AsSingle()` - делает найденный объект с нужным скриптом в каком-то смысле синглтоном. Теперь инсталлер не будет искать другие такие объекты, а всем желающим будет прокидывать только первый найденный.

Если поместить такой инсталлер на персистент сцену, а также поместить туда не синглтон объект. Найти его и указать после `AsSingle()` → `.NonLazy()`; то `Zenject` не будет дожидаться, когда кто-то попросит этот объект, а инициализирует его сразу. Когда будут подгружаться различные уровни игры и делать запрос к типу данных через атрибут `Inject` → им будет пробрасываться один и тот же глобальный объект, словно это экземпляр, а не полноценный синглтон. Это позволит в некоторых случаях проводить закрытые тесты. Такой объект будет хранить `ProjectContext` (данные экземпляра).